Aiden Munger

Zag Farkle - CPSC 224

Program Purpose:

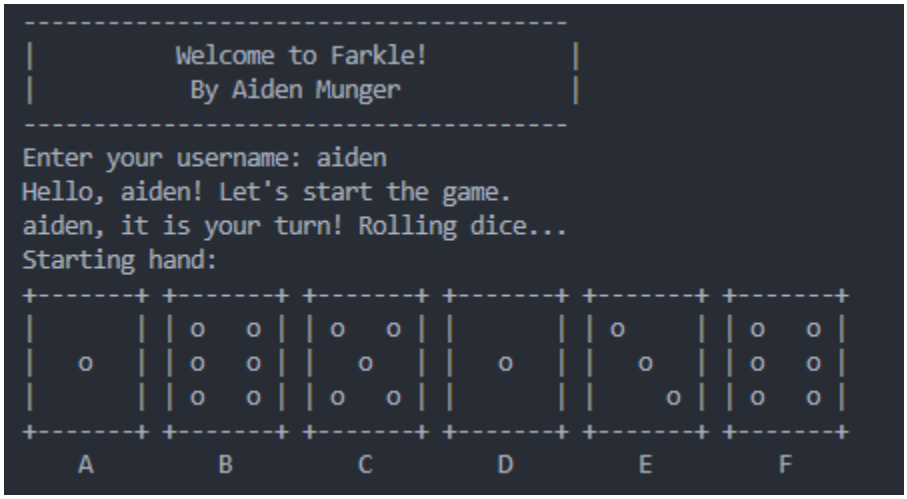
The purpose of Zag Farkle is to implement a playable version of the dice game Farkle using Java. The player roles six dice and attempts to create scoring combinations to earn points. These combinations include straights, three pairs, triples, and individual ones and fives. The player can select scoring dice to create a meld, reroll the remaining unused dice, or bank their points to end their turn. The program also handles situations such as Farkles, quitting, and Hot Hands. This assignment gave me an opportunity to practice object-oriented programming in Java, separate tasks between classes, and test my implementation of features.

Program Design:

I organized the program into multiple classes so each class has a specific responsibility. The Farkle class serves as the starting point and handles the initial program setup and player name. The Turn class manages the current turn, including banking points, rerolling dice, and determining when a turn ends. The Hand class keeps track of the dice and separates unused dice from dice that have been selected for a meld. The Dice class represents an individual die and handles rolling and displaying its value. The Meld class stores the dice selected for a scoring combination, while the Score class calculates the points earned from a meld. The Menu class provides the player with options during the turn.

Separating these responsibilities made the program easier to organize and maintain. For example, scoring calculations can be tested independently of the user interface, while the hand class manages which dice are available for the next roll. This design also makes it easier to identify and fix problems without having to rewrite the entire program.

Starting Screen:



Testing:

I created a ScoreTest class to check important parts of the program's behavior. One example is the three-pairs scoring test, which creates a meld containing the dice values 1, 1, 4, 4, 6, and 6. The test passes when Score.calculate() returns 750 points, which is the expected score for three pairs. This test

demonstrates that the scoring logic correctly identifies this combination instead of treating the dice as unrelated individual values.

I also tested other combinations, including a straight, a single one, three threes, and triple ones. Additionally, I tested Farkle detection and the handling of normal, blank, and whitespace-only player names. All nine checks passed, giving me confidence that these specific parts of the program behave as expected.

Major Design and Programming Challenges:

One of the main challenges was keeping track of the dice as they moved between the unused dice and the meld. When a player selects a die for a meld, it must no longer be available for rerolling. I addressed this by maintaining separate collections in the Hand class and moving dice between them when the player adds or removes a die from a meld. This approach keeps the state of the hand organized and makes it easier to determine which dice should be rerolled.

One of the main challenges was keeping track of the dice as they moved between the unused dice and the meld. When a player selects a die for a meld, it must no longer be available for rerolling. I addressed this by maintaining separate collections in the Hand class and moving dice between them when the player adds or removes a die from a meld. This approach keeps the state of the hand organized and makes it easier to determine which dice should be rerolled.

Retrospective:

One of the main challenges was keeping track of the dice as they moved between the unused dice and the meld. When a player selects a die for a meld, it must no longer be available for rerolling. I addressed this by maintaining separate collections in the hand class and moving dice between them when the player adds or removes a die from a meld. This approach keeps the state of the hand organized and makes it easier to determine which dice should be rerolled.